An introduction to Flutter

Worked example: a Hello World app built in eight steps

Key ideas

- Flutter is a collection of **widgets**: **everything is a widget**.
- A widget is a Dart class for one part of the screen (an icon, a button, a page).
- **Stateless** (`StatelessWidget`): fixed; a parent must build a new one to show something else. `Text`, `Icon`, buttons.
- **Stateful** (`StatefulWidget`): its contents change while the app runs; `setState` redraws it. `Checkbox`, `Radio`, `TextField`.

Stateless and stateful side by side

```
class Greeting extends StatelessWidget {
  final String name; // set once
  const Greeting({super.key,
                  required this.name});
  @override
  Widget build(BuildContext context) =>
      Text('Hello, $name');
}
// AcceptTerms is stateful: it keeps
// bool _accepted and flips it with
// setState(() => _accepted = value!)
```

Create a project and an emulator

Android Studio	File → New → New Flutter Project; check the SDK path; name `week4`
VS Code	`Ctrl+Shift+P` → **Flutter: New Project** → Application
Terminal	`flutter create week4`
Emulator	Android Studio **Device Manager** → Create device → Launch

The new project is a counter sample; your code is in `lib/main.dart`.

Step 1: the smallest app

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return Container(color: Colors.white);
  }
}
```

- `runApp` attaches a widget to the screen; it fills the screen.
- `const`: the widget never changes, so Flutter can reuse it (faster).
- Android Studio: type `stless` to create the class.
- An empty `Container()` is a black screen.

Steps 2 to 5: text, page, bar, style

```
MaterialApp(                        // step 2
  home: Scaffold(                   // step 3
    appBar: AppBar(                 // step 4
      title: const Text('Home Page'),
      centerTitle: true),
    body: const Center(
      child: Text('Hello World',
        style: TextStyle(           // step 5
          fontSize: 24,
          color: Colors.deepPurple)),
    ),
  ),
)
```

`MaterialApp`	Material look; `home` is the first screen
`Center`	puts its `child` in the middle
`Scaffold`	page structure; `body` is the content
`AppBar`	the bar at the top

Red letters with a yellow underline = text with no default style: the text is in a `MaterialApp`

but not inside a `Scaffold`. Add the `Scaffold`.

Step 6: your own widget

Extract it: Android Studio **Refactor** → **Extract Flutter Widget**; VS Code `Ctrl+.` → **Extract Widget**. Then add a parameter:

```
class MyTextWidget extends StatelessWidget {
  final String text;
  const MyTextWidget(
      {required this.text, super.key});
  @override
  Widget build(BuildContext context) =>
      Center(child: Text(text,
        style: TextStyle(fontSize: 24,
          color: Colors.deepPurple)));
}
// use: body: MyTextWidget(text: 'Text 1')
```

Steps 7 and 8: Column and Row

```
Column(children: [
  SizedBox(height: 16.0),
  MyTextWidget(text: 'Text 1'),
  SizedBox(height: 16.0),
  MyTextWidget(text: 'Text 2'),
  Row(
    mainAxisAlignment:
        MainAxisAlignment.spaceEvenly,
    children: [Text('Text 3'),
      Text('Text 4'), Text('Text 5')],
  ),
])
```

`Column`: vertical; `Row`: horizontal; both take a *list* in `children`. `SizedBox`: a fixed empty space.

Copyrighted content Dr Mohamad AOUDE

Logical pixels

- All Flutter sizes (`16.0`, `fontSize: 24`) are **logical pixels**.
- physical pixels = logical pixels × **device pixel ratio**.
- So a widget keeps the same size on every screen.

```
final size  = MediaQuery.sizeOf(context);
final ratio =
    MediaQuery.devicePixelRatioOf(context);
```

```
Text('${size.width} x ${size.height}');
```

More AppBar options

```
AppBar(
  title: const Text('AppBar options'),
  backgroundColor: Colors.teal,
  foregroundColor: Colors.white,
  elevation: 4,          // 0 = flat
  leading: IconButton(   // left end
    icon: const Icon(Icons.menu),
    onPressed: () => _say(context, 'Menu')),
  actions: [ /* IconButtons, right end */ ],
)
void _say(BuildContext context, String m) =>
  ScaffoldMessenger.of(context)
    .showSnackBar(SnackBar(content: Text(m)));
```

Card, Padding and Expanded

- `Card`: rounded rectangle with a small shadow, to group content.
- `Padding`: space around its child.
- `Expanded` (in a `Column` or `Row`): fills the space the other children leave free; `flex` sets the share.

```
Column(children: [
  Container(height: 60, color: grey),
  Expanded(flex: 1, child: red),   // 1/4
  Expanded(flex: 2, child: green), // 1/2
  Expanded(flex: 1, child: blue),  // 1/4
])
```

You never compute a size: Flutter shares the free space on any screen.

Widgets of this lecture

`Container`	a box with a color and a size
`MaterialApp`	root of a Material app
`Scaffold`	structure of a page
`AppBar`	bar at the top
`Center`	centers its child
`Text`	a line of text; `style`
`Column` / `Row`	children vertical / horizontal
`SizedBox`	fixed empty space
`IconButton`	tappable icon; `onPressed`
`Card`	grouped content with a shadow
`Padding`	space around the child
`BottomAppBar`	bar in `bottomNavigationBar`
`Expanded`	fills free space; `flex`

Good habits

- Write `const` before widgets that never change.
- Turn repeated widgets into your own class with parameters.
- One `Scaffold` per page; never one inside another.

Copyrighted content Dr Mohamad AOUDE